Practical 6

Programs on Exceptions and asynchronous programming

Exceptions

Exceptions in **C# .NET Core** are runtime errors that occur while a program is executing, such as dividing by zero, invalid user input, accessing a null object, or reading a missing file. .NET Core provides a structured exception-handling mechanism using the try, catch, finally, and throw keywords to detect, handle, and recover from these errors without abruptly terminating the application. Proper exception handling improves application reliability, helps display meaningful error messages, ensures resources are released correctly, and makes debugging and maintenance easier.

Asynchronous programming

Asynchronous programming in **C# .NET Core** is a programming technique that allows a program to perform long-running tasks, such as reading files, accessing databases, or making web requests, without blocking the main thread. It uses the async and await keywords along with the Task class to execute operations efficiently while keeping the application responsive. Instead of waiting for one task to finish before starting another, asynchronous programming allows other work to continue, improving performance and user experience. It is widely used in .NET Core applications, especially in web applications, APIs, and desktop applications where responsiveness and scalability are important.

Program 1: Handling Divide by Zero Exception

```
using System;

namespace ExceptionDemo
{
   class Program
   {
      static void Main(string[] args)
      {
         int a = 20;
         int b = 0;

         try
         {
            int result = a / b;
            Console.WriteLine("Result = " + result);
         }
         catch (DivideByZeroException ex)
         {
            Console.WriteLine("Exception: " + ex.Message);
         }
```

```
        finally
        {
            Console.WriteLine("Finally block executed.");
        }

        Console.ReadLine();
    }
  }
}
```

Output:

Exception: Attempted to divide by zero.
Finally block executed.

Program 2: Multiple Catch Blocks

```
using System;

namespace ExceptionDemo
{
  class Program
  {
    static void Main(string[] args)
    {
      try
      {
        Console.Write("Enter first number: ");
        int a = int.Parse(Console.ReadLine());

        Console.Write("Enter second number: ");
        int b = int.Parse(Console.ReadLine());

        int result = a / b;

        Console.WriteLine("Result = " + result);
      }
      catch (FormatException)
      {
        Console.WriteLine("Please enter numeric values.");
      }
      catch (DivideByZeroException)
      {
        Console.WriteLine("Cannot divide by zero.");
      }
      catch (Exception ex)
      {
        Console.WriteLine(ex.Message);
      }

      Console.ReadLine();
```

```
        }
    }
}
```

Output

```
Enter first number: 20
Enter second number: 0
Cannot divide by zero.
```

Program 3: Custom Exception

```
using System;

class AgeException : Exception
{
    public AgeException(string message) : base(message)
    {
    }
}

class Program
{
    static void Main()
    {
        int age = 15;

        try
        {
            if (age < 18)
            {
                throw new AgeException("Age must be 18 or above.");
            }

            Console.WriteLine("Eligible");
        }
        catch (AgeException ex)
        {
            Console.WriteLine(ex.Message);
        }

        Console.ReadLine();
    }
}
```

Output:

```
Age must be 18 or above.
```

Program 4: Basic Asynchronous Programming using async and await

```
using System;
using System.Threading.Tasks;

class Program
{
   static async Task Main(string[] args)
   {
      Console.WriteLine("Process Started...");

      await DoWorkAsync();

      Console.WriteLine("Process Completed.");

      Console.ReadLine();
   }

   static async Task DoWorkAsync()
   {
      Console.WriteLine("Working...");

      await Task.Delay(3000);

      Console.WriteLine("Work Finished.");
   }
}
```

Output:

```
Process Started...
Working...
(Waits for 3 seconds)
Work Finished.
Process Completed.
```

Program 5: Download Simulation Using Async

```
using System;
using System.Threading.Tasks;

class Program
{
   static async Task Main()
   {
      Console.WriteLine("Downloading file...");

      string file = await DownloadFileAsync();

      Console.WriteLine(file);
```

```
        Console.ReadLine();
    }

    static async Task<string> DownloadFileAsync()
    {
        await Task.Delay(5000);

        return "Download Completed Successfully.";
    }
}
```

Output:

Downloading file...
(Waits for 5 seconds)
Download Completed Successfully.

Program 6: Running Multiple Tasks

```
using System;
using System.Threading.Tasks;

class Program
{
    static async Task Main()
    {
        Task task1 = Task.Run(() =>
        {
            Console.WriteLine("Task 1 Started");
            Task.Delay(2000).Wait();
            Console.WriteLine("Task 1 Completed");
        });

        Task task2 = Task.Run(() =>
        {
            Console.WriteLine("Task 2 Started");
            Task.Delay(1000).Wait();
            Console.WriteLine("Task 2 Completed");
        });

        await Task.WhenAll(task1, task2);

        Console.WriteLine("All Tasks Completed.");

        Console.ReadLine();
    }
}
```

Output:-

```
Task 1 Started
```

Task 2 Started
Task 2 Completed
Task 1 Completed
All Tasks Completed.

Program 7: Reading User Input with Exception Handling

```
using System;

class Program
{
   static void Main()
   {
      try
      {
         Console.Write("Enter your age: ");
         int age = Convert.ToInt32(Console.ReadLine());

         Console.WriteLine("Your Age is: " + age);
      }
      catch (FormatException)
      {
         Console.WriteLine("Invalid input. Enter numbers only.");
      }

      Console.ReadLine();
   }
}
```

Output:-

Enter your age: ABC
Invalid input. Enter numbers only.